
Exception Handling

Termination:

Termination is the process of stopping or ending the execution of a program.

Types of Termination

1. Normal Termination

Normal termination occurs when the program runs successfully without any errors and completes all statements.

Example Program

```
1.print(100)
2.print(200)
3.print(300)
4.print(400)
5.print(500)
```

Step 1

→ print(100) executes => Output: 100

Step 2

→ print(200) executes => Output: 200

Step 3

→ print(300) executes => Output: 300

Step 4

→ print(400) executes => Output: 400

Step 5

→ print(500) executes => Output: 500

After the last statement, it comes out, showing the program termination.

Final Output

```
100
200
300
400
500
```

2. Forced Termination

Forced termination means the program stops execution immediately before completing all statements.

In Python, we can forcefully stop a program using the `exit()` function.

`exit()` : is used to terminate the program immediately, even if there are more statements remaining in the program

Example Program

```
1.print(100)
2.print(200)
3.exit()
4.print(300)
5.print(400)
6.print(500)
```

Step 1

→ `print(100)` executes => Output: 100

Step 2

→ `print(200)` executes => Output: 200

Step 3

→ `exit()`

> When `exit()` is encountered, the program terminates immediately.

> Remaining statements are skipped.

3. Abrupt Termination

Abrupt termination means the program stops suddenly in the middle of execution due to an mistake.

Example Program

```
1.print(100)
2.print(200)
3.print(300)
4.print(400)
5.print(500)
```

Python first checks the entire program for syntax errors before execution.

Step 1 => Python reads the program.

Step 2 => It finds an error in this line:

```
print(400
```

The closing parenthesis) is missing.

Step 3 => Because of this `SyntaxError`, the program will not execute any statement.

Error Message: `SyntaxError: '(' was never closed`

Error:

An error is a mistake in a program that causes the program to stop execution or produce wrong(incorrect) output.

Types of Errors

1. Compile Time Error
2. Runtime Error
3. Logical Error

1.Compile Time Error

> A compile time error is an error detected before the execution of a program due to incorrect syntax.

>Python checks the syntax of the entire program first, and if it finds a mistake, the program will not execute.

Example 1:

```
print("Hello
```

Output: `SyntaxError: unterminated string literal`

Example 2:

```
def display():  
    print("Hello")  
display()
```

Output: IndentationError: expected an indented block

2.Runtime Errors

A Runtime Error is an error that occurs during the execution of a program after the program has started running.

Ex 1. ZeroDivisionError

Occurs when a number is divided by zero.

```
a = 10  
b = 0  
print(a/b)
```

Output: ZeroDivisionError

Ex 2. NameError

Occurs when a variable is used without defining it.

```
print(x)
```

Output: NameError

Ex 3. KeyError

Occurs when a dictionary key does not exist.

```
d = {"a":10}  
k = eval(input("enter the key: "))  
print(d[k])
```

Output: KeyError

Ex 4. TypeError

Occurs when different data types are used in an operation.

```
print(10 + "20")
```

Output: TypeError

Ex 5. IndexError

Occurs when an invalid index is used in a list or tuple.

```
l = [10,20,30]  
print(l[5])
```

Output: IndexError

Ex 6. AttributeError

Occurs when an object does not support a particular method.

```
x = 10  
x.append(5)
```

Output: AttributeError

Ex 7. FileNotFoundError

Occurs when Python cannot find the specified file.

```
f = open("data.txt")
```

Output: FileNotFoundError

Ex 8. FileExistsError

Occurs when trying to **create a file that already exists using 'x' mode.**

```
f = open("data.txt","x")
```

Output: FileExistsError

3.Logical Errors:

A Logical Error occurs when the program runs successfully without any error message but produces incorrect output.

The mistake is in the logic of the program, not in the syntax.

Ex 1: Average of First 3 Natural Numbers**Wrong Program (Logical Error)**

```
print( 1 + 2 + 3 / 3)
```

Output: 4.0

The program runs successfully, but the logic is wrong because division should be applied to the sum of all numbers.

Correct Program

```
print( ( 1 + 2 + 3 ) / 3)
```

Output: 2.0

Ex 2: Even Number Checking**Wrong Program (Logical Error)**

```
n = 10
if n % 2 == 1:
    print("Even Number")
else:
    print("Odd Number")
```

Output: Odd Number

The program runs successfully but logic is wrong because even numbers have remainder 0.

Correct Program

```
n = 10
if n % 2 == 0:
    print("Even Number")
else:
    print("Odd Number")
```

Output: Even Number

Exception

An Exception is an error that occurs during the execution of a program and interrupts the normal flow of the program.

There are 2 types:

1. Built-in Exceptions
 2. Custom or user-defined Exceptions
-

1. Built-in Exceptions

- > Built-in exceptions are predefined exceptions provided by Python.
- > They occur automatically when an error happens during program execution.

Ex:

1. ZeroDivisionError
2. NameError
3. KeyError
4. TypeError
5. IndexError
6. AttributeError
7. FileNotFoundError
8. FileExistsError

When these exception occurs, the program stops execution unless it is handled using exception handling.

Hence we need to handle it using exception handlers.

Exception Handlers

- Exception handlers are blocks of code designed to catch and manage exceptions that occur during program execution.
 - They allow programs to handle errors gracefully instead of crashing suddenly.
 - A program may stop working or terminate unexpectedly due to an error, but exception handlers help control such situations.
 - In Python, exceptions are handled using the try and except blocks.
 - The **except block acts as the exception handler**.
 - It catches the error and allows the program to continue execution.
-

General Syntax of Exception Handling (try – except – else – finally)

try:

code that may generate an exception(doubtfull codes)

except :

code to handle the exception(handler codes)

else:

executes if no exception occurs in try block

finally:

executes always whether exception occurs or not

Ways to handle the exceptions

1.Normal Way of handling the exception

Example:

try:

n = int(input("Enter a number: "))

except :

print("Invalid input")

else:

print("You entered:", n)

2. Handling the Specific exception

Syntax:

try:

 # doubtful code

except ExceptionType:

 # handler code

Example:

```
print("Execution started")
```

try:

```
    num = int(input("Enter an integer: "))
```

except ValueError:

```
    print("Invalid input! Please enter an integer value.")
```

else:

```
    print("You entered:", num)
```

finally:

```
    print("Program execution completed.")
```

3. Providing alias name for exception

Syntax:

try:

 # doubtful code

except ExceptionType as alias_name:

 # handler code

Example:

try:

```
    num = 10
```

```
    num.append(5) # integers do not have append() method
```

except AttributeError as e:

```
    print("Attribute Error occurred:", e)
```

else:

```
    print("No error occurred")
```

4. Handling multiple exceptions using multiple except blocks

Syntax:

```
try:
    # doubtful code
except ExceptionType1 as alias1:
    # handler code for exception 1
except ExceptionType2 as alias2:
    # handler code for exception 2
.
.
except ExceptionType n as alias n:
    # handler code for exception n
```

Example:

```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
    res = num1 / num2
except ValueError as ve:
    print("Value Error:", ve)
except ZeroDivisionError as zde:
    print("Zero Division Error:", zde)
else:
    print("Result:", res)
finally:
    print("Program execution completed")
```

5. Handling multiple exceptions in a single line

Syntax:

```
try:
    # doubtful code
except (ExceptionType1, ExceptionType2, ..., ExceptionType n) as alias_name:
    # handler code
```

Example:

```
print("Execution Started")
try:
    list = [10, 20, 30]
    index = int(input("Enter index value: "))
    res = list[index]
except (IndexError, ValueError) as e:
    print(e)
else:
    print(res)
finally:
    print("Program execution completed")
```

6. Handling Exceptions using Exception class**Syntax:**

```
try:
    # doubtful code
except Exception as alias_name:
    # handler code
```

Example: Here there is a chance of getting NameError and KeyError

```
print("Execution Started")
d = {1: 2, "name": "Pyspiders" , 3.5: "Orange"}
try:
    key = eval(input("Enter the key: "))
    res = d[key]
except Exception as e:
    print(e)
else:
    print("Key found successfully and its value is: ",res)
finally:
    print("Execution completed")
```

Exception class :

>It is the base class for most built-in exceptions in Python.

>It is used to handle different types of runtime errors in a general way.

7. Using raise Keyword and Handling the Error**Example:**

```
print("Execution Started ")
try:
    age = float(input("Enter age: "))
    if age < 0:
        raise ValueError("Age cannot be negative")
except ValueError as e:
    print("Error:", e)
else:
    print("Valid age entered", age)
finally:
    print("Program execution completed")
```

__str__() Method in Python

__str__() is a special method (magic method) in Python used to return a readable string representation of an object.

Example 1:

```
class MYCLASS:
    def __str__(self):
        return "I am class called MYCLASS"

obj = MYCLASS()

print(obj)

# I am class called MYCLASS
```

Example 2:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"Name: {self.name}, Age: {self.age}"

s1 = Student("John", 20)
print(s1)

#Name: John, Age: 20
```

`__str__()` Method Key-Points

1. `__str__()` must return a string.
 2. It is automatically called when we use `print(obj)` or `str(obj)`.
 3. Inside `__str__()` we should not use `print()`; only return should be used.
 4. If `__str__()` returns anything other than a string, Python raises a `TypeError`.
 5. `__str__()` takes only one parameter, which is `self`.
 6. `print(obj)` internally uses the value returned by `__str__()` to display the object.
-

2.Custom Exceptions:

>Custom exceptions are user-defined exceptions created by the programmer to handle specific errors according to program requirements.

>Python provides many built-in exceptions, but sometimes we need our own exceptions for particular conditions in a program.

How to Create a Custom Exception

To create a custom exception, we need to create a class that inherits from the Exception class.

Syntax

```
class CustomExceptionName(Exception):
```

```
    pass
```

- CustomExceptionName → user-defined exception class
- Exception → base class
- pass → placeholder if no extra code is needed

Steps to Perform Custom Exception:

Step 1: Create a Custom Exception Class.

- Create a class that inherits from the Exception class.

Step 2: Raise the Exception:

- Use the raise keyword to trigger the custom exception when a condition fails.

Step 3: Handle the Exception.

- Use a try and except block to handle the exception.

Custom Exceptions Can Be Implemented Using:

1. Function
 2. Class
 3. Custom Attributes
-

Example 1 : Custom Exception using Function(creating and raising)**#step 1: Create a Custom Exception Class**

```
class NegativeAgeError(Exception):  
    pass  
  
def check_age(age):  
    if age < 0:  
        #Step 2: Raise the Exception  
        raise NegativeAgeError("Age cannot be negative")  
    else:  
        print("Valid Age:",age)  
user_age = float(input("enter the age: "))  
check_age(user_age)
```

Custom Exception using Function(create + raise + handling)

```
class NegativeAgeError(Exception):  
    pass  
  
def check_age(age):  
    if age < 0:  
        raise NegativeAgeError("Age cannot be negative")
```

Step 3: Handling the Exception.

```
try:  
    user_age = int(input("Enter age: "))  
    check_age(user_age)  
except NegativeAgeError as e:  
    print("Custom Exception:", e)  
else:  
    print("Valid Age: ",user_age)
```

Example 2 : Custom Exception using another class**Approach1 :****# Step 1: Create custom exception**

```
class NameInvalidError(Exception):  
    pass
```

Another class to validate name

```
class PERSON:
```

#normal instance method

```
def check_name(self, name):  
    if not name.isalpha():
```

#Step 2: Raise the Exception

```
        raise NameInvalidError("Name should contain only alphabets")
```

Step 3: Handling the exception

```
try:
```

```
    obj_p = PERSON()  
    user_name = input("Enter name: ")  
    obj_p.check_name(user_name)
```

```
except NameInvalidError as e:
```

```
    print("Custom Exception:", e)
```

```
else:
```

```
    print("Valid Name:",user_name)
```

Approach 2 :

```
class InvalidNameError(Exception):
    pass

class check_name:
    def __init__(self, name):
        self.name = name

user_name = input("enter the name: ")
obj = check_name(user_name)

try:
    for i in user_name:
        if not ("A" <= i <= "Z" or "a" <= i <= "z"):
            raise InvalidNameError("Name should contain only alphabets")

except InvalidNameError as e:
    print(e)

else:
    print("Valid name:", user_name)
```

Approach 3 :

```
class InvalidNameError(Exception):
    pass

class check_name:
    def __init__(self, name):
        self.name = name

    def validate_name(self):
        for i in self.name:
            if not ("A" <= i <= "Z" or "a" <= i <= "z"):
                raise InvalidNameError("Name should contain only alphabets")

user_name = input("Enter the name: ")
try:
    obj = check_name(user_name)
    obj.validate_name()

except InvalidNameError as e:
    print(e)

else:
    print("Valid name:", obj.name)
```

Example 3 : Custom Exception using custom attributes**# Step 1: Create custom exception with attributes**

```
class NameInvalidError(Exception):
```

```
    def __init__(self, name, msg):
        self.name = name
        self.msg = msg
```

Step 2 & 3: Raise and handle the exception

```
try:
```

```
    user_name = input("Enter name: ")
```

```
    if not (user_name.isalpha()):
```

```
        raise NameInvalidError(user_name, "Name should contain only alphabets")
```

```
except NameInvalidError as e:
```

```
    print("Invalid Name:", e.name)
```

```
    print("Reason:", e.msg)
```

```
else:
```

```
    print("Valid Name:", user_name)
```

Example 4 : Custom Exception using custom attributes with __str__()**# Step 1: Create custom exception**

```
class WeakPasswordError(Exception):  
    def __init__(self, password):  
        self.password = password  
  
    def __str__(self):  
        return f"Password '{self.password}' is too weak. Min 6 characters  
        required."
```

Step 2 & 3: Raise and handle the exception

```
try:  
    user_password = input("Enter password: ")  
    if len(user_password) < 6:  
        raise WeakPasswordError(user_password)  
except WeakPasswordError as e:  
    print("Custom Exception:", e)  
else:  
    print("Strong Password")
```

- Printing the exception automatically calls the __str__() method.
-
-

Nested try and except block:

A try-except block placed inside another try or except block to handle different exceptions.

Syntax:**try:**`# outer block``# some code`**try:**`# inner block``# some code`**except InnerExceptionType:**`# handle inner exception`**except OuterExceptionType:**`# handle outer exception`**Example:****try:**`x = int(input("Enter numerator: "))``y = int(input("Enter denominator: "))`**try:**`res = x / y`**except ZeroDivisionError:**`print("Inner block: Cannot divide by zero")`**else:**`print("Result is:", result)`**except ValueError:**`print("Outer block: Invalid input, please enter integers")`
